OS-ORANGE-UNIT4

NAME: AHIDWISH JAVALI SECTION: 4A SRN: PES1UG24CS037

PHASE1: Object Storage Foundation

1A:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

1B:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ find .pes/objects -type f
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

PHASE2: Tree Objects

2A:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

2B:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ xxd .pes/objects/10/bef684865401ccdde603e94bb12f922a476c3c763a0eb3a287
65a6f7ee0cc1 | head -20
00000000: 7472 6565 2034 3800 3130 3036 3434 2074  tree 48.100644 t
00000010: 6573 742e 7478 7400 ae8d 817b a2bf 5226  est.txt....{..R&
00000020: 78bc f4d7 222d e2fa 92db 28f2 c65d ca4f  x..."-....(..].O
00000030: 8312 921d 6eff 22b5                      ....n.".
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

PHASE3: The Index (Staging Area)

3A:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./pes add test.txt
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./pes status
Staged changes:
  staged:     test.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  Makefile
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  object.c
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  pes.h
  untracked:  .gitignore
  untracked:  README.md
  untracked:  pes.c
  untracked:  commit.h
  untracked:  index.h
  untracked:  test_tree.c
  untracked:  tree.c
  untracked:  tree.h

ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

3B:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ cat .pes/index
100644 b4563769bac79167c288d4f1b6eac6f42b713b639eb65ac41eb183e35f0d1181 1776677333 12 test.txt
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

PHASE4: Commits and History

4A:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ ./pes log
commit 784fc01e2c5da061f84ba1998d16de710911c47b85e03356b63bcfb2a65d9a20
Author: PES User <pes@localhost>
Date:   1776683394

    third commit

commit 450d44d013798a72a7cd472e0c6d395e0a491d735a7cae4d9299730081e5a67d
Author: PES User <pes@localhost>
Date:   1776682676

    second commit

commit 2f486e221b6705ff2c9419c0168a6ef4475235b5a852d28d4fe7a798c665cf15
Author: PES User <pes@localhost>
Date:   1776682373

    first commitirst commit<Q

ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

4B:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/10/bef684865401ccdde603e94bb12f922a476c3c763a0eb3a28765a6f7ee0cc1
.pes/objects/2f/486e221b6705ff2c9419c0168a6ef4475235b5a852d28d4fe7a798c665cf15
.pes/objects/45/0d44d013798a72a7cd472e0c6d395e0a491d735a7cae4d9299730081e5a67d
.pes/objects/78/4fc01e2c5da061f84ba1998d16de710911c47b85e03356b63bcfb2a65d9a20
.pes/objects/86/de9c0f6473ee5d259b4768f33139b4e5dc70e3920b07596abd3d5bc69164b7
.pes/objects/ae/8d817ba2bf522678bcf4d7222de2fa92db28f2c65dca4f8312921d6eff22b5
.pes/objects/b4/563769bac79167c288d4f1b6eac6f42b713b639eb65ac41eb183e35f0d1181
.pes/objects/f6/b94c988136d131c539ea930c1b3fb028a9fb254c24a6598be8a22b93220b4b
.pes/refs/heads/main
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

4C:

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ cat .pes/refs/heads/main
2f486e221b6705ff2c9419c0168a6ef4475235b5a852d28d4fe7a798c665cf15
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$
```

```
ahidwish@ahidwish-VirtualBox:~/Desktop/PES1UG24CS037-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

ANSWER: To implement pes checkout <branch>, the main change inside .pes/ is updating the HEAD file so that it points to the selected branch, i.e., ref: refs/heads/<branch>. Then, the commit hash stored in that branch file must be read, and its corresponding tree object must be used to reconstruct the working directory. This means deleting or updating existing files and recreating them exactly as stored in the target commit's tree. The index should also be updated to match this new state. The operation becomes complex because it is not just metadata changes — it involves modifying actual files on disk. Handling nested directories, file deletions, and ensuring no data loss during the switch adds to the complexity.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

ANSWER: To detect a dirty working directory, we can compare the current working directory with the index. The index already stores metadata like file size and modification time, so we can quickly identify if a tracked file has been modified since it was last staged. If a file is modified in the working directory and also differs in the target branch's tree, then switching branches would overwrite those changes, which must be prevented. So, before checkout, we check for any such conflicts by comparing index entries with both the current working directory and the target tree from the object store. If any conflicting file is found, the checkout operation should stop and notify the user.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

ANSWER: In a detached HEAD state, the HEAD file contains a direct commit hash instead of pointing to a branch. If a user makes new commits in this state, those commits are created normally, but they are not referenced by any branch. As a result, they can become “lost” once the user switches to another branch, since no pointer keeps track of them. However, these commits are still present in the object store. A user can recover them if they remember the commit hash (for example, from logs) by creating a new branch pointing to that commit. If not recovered, they may eventually be removed by garbage collection.

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

ANSWER: To remove unreachable objects, we start from all references such as branch heads and traverse the commit graph. From each commit, we follow its parent links and associated tree objects, and from trees we follow all blob and subtree references. This traversal marks all reachable objects. A hash set can be used to efficiently store and check these visited object IDs. After marking, we scan the entire object store and delete any object whose hash is not in the reachable set. In a repository with around 100,000 commits and 50 branches, we would not visit all commits repeatedly — many branches share history — so the total number of unique objects visited would roughly be on the order of the total commits plus their trees and blobs, not 50 times that number.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

ANSWER: Running garbage collection at the same time as a commit operation is risky because of race conditions. For example, while a new commit is being created, its objects (like blobs or trees) may have been written but not yet referenced by any branch. If garbage collection runs at that exact moment, it might consider those objects unreachable and delete them, even though the commit process is about to reference them. This leads to corrupted history. Git avoids this issue by ensuring that objects are not immediately deleted and by using mechanisms like temporary references and grace periods. It also avoids running aggressive garbage collection while writes are in progress, ensuring that newly created objects are safely linked before any cleanup happens.

Final integration: (make-integration)

=== Running integration tests ===

```
bash test_sequence.sh
=== PES-VCS Integration Test ===
--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists
--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt
Unstaged changes:
  (nothing to show)
Untracked files:
  (nothing to show)
--- First Commit ---
DEBUG: start commit_create
DEBUG: step 1 - building tree
DEBUG: step 2 - reading HEAD
DEBUG: no parent (first commit)
DEBUG: step 3 - setting author
DEBUG: step 4 - setting message
DEBUG: step 5 - serializing commit
DEBUG: step 6 - writing object
DEBUG: step 7 - updating HEAD
DEBUG: SUCCESS - commit created
Committed: 6f1cf4c89eda... Initial commit
Log after first commit:
commit 6f1cf4c89eda21304de0deb1e186d0e0e141182b37197990b50e34802feff033
Author: PES User <pes@localhost>
Date:   177668527
```

```
    Initial commit
--- Second Commit ---
DEBUG: start commit_create
DEBUG: step 1 - building tree
DEBUG: step 2 - reading HEAD
DEBUG: parent found
DEBUG: step 3 - setting author
DEBUG: step 4 - setting message
DEBUG: step 5 - serializing commit
DEBUG: step 6 - writing object
DEBUG: step 7 - updating HEAD
DEBUG: SUCCESS - commit created
Committed: 8e2bcad6b59c... Update file.txt
--- Third Commit ---
DEBUG: start commit_create
DEBUG: step 1 - building tree
DEBUG: step 2 - reading HEAD
DEBUG: parent found
DEBUG: step 3 - setting author
DEBUG: step 4 - setting message
DEBUG: step 5 - serializing commit
DEBUG: step 6 - writing object
DEBUG: step 7 - updating HEAD
DEBUG: SUCCESS - commit created
Committed: 4f51d46e4ec4... Add farewell
--- Full History ---
commit 4f51d46e4ec4d5bbe787c052b62f01985e79f30b270b70aa7511e089cb8e78a3
Author: PES User <pes@localhost>
Date:   1776685275

    Add farewell
commit 8e2bcad6b59c8130bf1eda12dc55260ebbaf1dbefac73eb8ac479946a1f1ada2
```

```
Author: PES User <pes@localhost>
Date:   1776685275
    Update file.txt
commit 6f1cf4c89eda21304de0deb1e186d0e0e141182b37197990b50e34802feff033
Author: PES User <pes@localhost>
Date:   1776685275
    Initial committial commiQ
--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
4f51d46e4ec4d5bbe787c052b62f01985e79f30b270b70aa7511e089cb8e78a3
--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/4f/51d46e4ec4d5bbe787c052b62f01985e79f30b270b70aa7511e089cb8e78a3
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/6f/1cf4c89eda21304de0deb1e186d0e0e141182b37197990b50e34802feff033
.pes/objects/8e/2bcad6b59c8130bf1eda12dc55260ebbaf1dbefac73eb8ac479946a1f1ada2
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
=== All integration tests completed ===
```